HDOC DAY-43

Title: Advanced String Processing: Longest Word Extraction & String Rotation Validation

1. Problem Statement

Write a menu-driven C program to perform the following operations using separate user-defined functions:

1. Find the longest word in a sentence.
2. Check if one string is a rotation of another.

2. Theoretical Background & Algorithm Logic

- **Longest Word in a Sentence:**

The function uses strtok() to break the input sentence into individual tokens separated by spaces or delimiters. Each token's length is computed via strlen(), and the longest token is tracked and stored via strcpy().

- **String Rotation Check:**

Two strings, str1 and str2, are rotations of each other if they are of equal length and str2 is a substring of str1 concatenated with itself (str1 + str1). This is verified efficiently using strcat() and strstr().

3. Test Case Table

Test Case	Selected Option	Input Data	Expected Output	Actual Output	Status
1	Longest Word	"Programming in C is amazing"	"Programming" (Length: 11)	"Programming" (Length: 11)	Pass
2	Rotation Check	str1 = "waterbottle", str2 = "erbottlewat"	Is a rotation	Is a rotation	Pass
3	Rotation Check	str1 = "hello", str2 = "world"	Not a rotation	Not a rotation	Pass
4	Exit	Choice 3	Program terminat	Program terminat	Pass

Test Case	Selected Option	Input Data	Expected Output	Actual Output	Status
			es cleanly	es cleanly	

4. Source Code Listing

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>

void read_string(const char *prompt, char str[], int max_size) {
   printf("%s", prompt);
   scanf(" %[^\n]", str);
}

void find_longest_word(const char sentence[], char longest_word[]) {
   char temp[500];
   strcpy(temp, sentence);
   longest_word[0] = '\0';
   int max_len = 0;

   char *token = strtok(temp, " \t\n\r");
   while (token != NULL) {
      int len = (int)strlen(token);
      if (len > max_len) {
         max_len = len;
         strcpy(longest_word, token);
      }
      token = strtok(NULL, " \t\n\r");
   }
}

bool are_rotations(const char str1[], const char str2[]) {
```

```
    int len1 = (int)strlen(str1);
    int len2 = (int)strlen(str2);
    if (len1 != len2 || len1 == 0) return false;

    char combined[1050];
    strcpy(combined, str1);
    strcat(combined, str1);

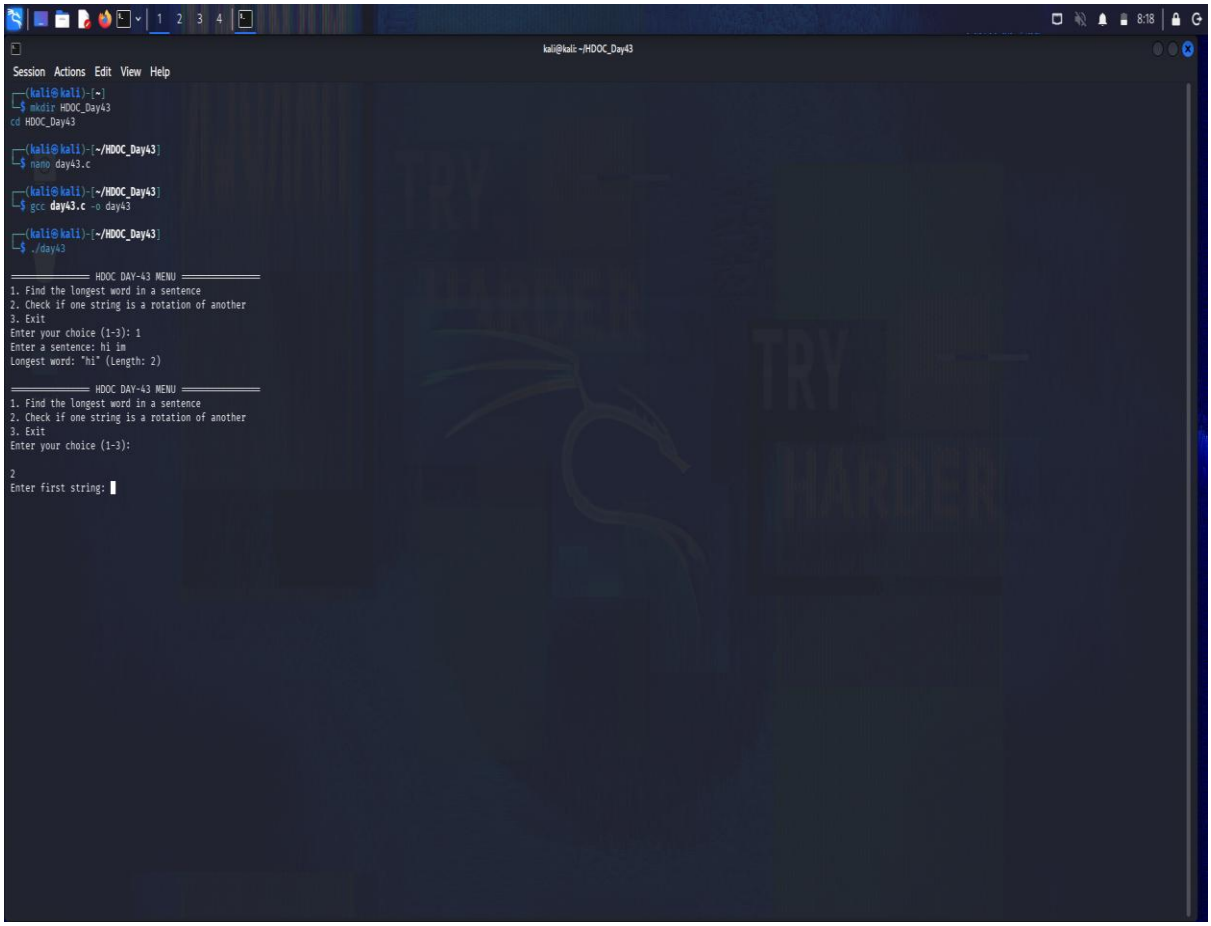
    return strstr(combined, str2) != NULL;
}

int main(void) {
    char sentence[500], longest[100], str1[100], str2[100];
    int choice;

    do {
        printf("\n1. Longest Word\n2. Rotation Check\n3. Exit\nChoice: ");
        if (scanf("%d", &choice) != 1) break;

        switch (choice) {
            case 1:
                read_string("Enter sentence: ", sentence, sizeof(sentence));
                find_longest_word(sentence, longest);
                printf("Longest word: %s\n", longest);
                break;
            case 2:
                read_string("Enter string 1: ", str1, sizeof(str1));
                read_string("Enter string 2: ", str2, sizeof(str2));
                if (are_rotations(str1, str2)) printf("ARE rotations.\n");
                else printf("NOT rotations.\n");
                break;
            case 3: printf("Exiting...\n"); break;
            default: printf("Invalid choice!\n"); break;
        }
    } while (choice != 3);

    return 0;
}
```


kali@kali: ~/HDOC_Day43
Session Actions Edit View Help
(kali㉿kali)-[~]
$ mkdir HDOC_Day43
cd HDOC_Day43
(kali㉿kali)-[~/HDOC_Day43]
$ nano day43.c
(kali㉿kali)-[~/HDOC_Day43]
$ gcc day43.c -o day43
(kali㉿kali)-[~/HDOC_Day43]
$ ./day43
═══════ HDOC DAY-43 MENU ═══════
1. Find the longest word in a sentence
2. Check if one string is a rotation of another
3. Exit
Enter your choice (1-3): 1
Enter a sentence: hi im
Longest word: "hi" (Length: 2)
═══════ HDOC DAY-43 MENU ═══════
1. Find the longest word in a sentence
2. Check if one string is a rotation of another
3. Exit
Enter your choice (1-3):
2
Enter first string: